

BuildWatch — документация проекта

Версия: 29 сентября 2026 года. Прототип мониторинга строительных площадок: датированные снимки камер, календарный план, детекция техники и проверяемые сигналы инспектору.

Команда: Басков Константин — разработчик; Манаков Дмитрий — продуктовый менеджер. Публичный прототип: <https://buildwatch.baski.pro/>. Репозиторий: <https://github.com/BaskovKonstantin/BuildWatch>.

1. Назначение и границы

BuildWatch связывает объект, датированный снимок, распознанную технику и действующий на дату снимка этап работ. Правила выдают сигнал, если обязательная техника не обнаружена, наблюдаемый класс не соответствует этапу, техника попала в опасную зону кадра или по серии снимков не подтверждена активность. Карточка объекта дополнительно считает прогноз по графику, динамику за 30 дней и долю вердиктов инспектора.

В интерфейсе: портфель объектов (карта и список), карточка площадки, календарный план, снимки с рамками, зоны кадра, предупреждения, ИИ-помощник и отчёт. Сигнал и прогноз требуют проверки человеком: один ракурс не доказывает отсутствие техники на всей площадке.

Главный экран открывается картой Москвы (Leaflet / OpenStreetMap). Цвет маркера: красный — открытая проблема, янтарный — нужна проверка, зелёный — открытых проблем нет. Справочник ЛТЦ задаёт виды работ без дат; календарь заполняется вручную или импортом CSV/XLSX с колонками «Этап», «Дата начала», «Дата окончания» (предпросмотр, затем подтверждение).

2. Архитектура и структура кода

Пользователь/камера → снимок + дата → FastAPI → SQLite/PostgreSQL

↓

очередь detection_jobs

↓

worker → YOLO (+ SAHI) → detections

↓

дата снимка + stages → rules → warnings

↓

Next.js: карточка, проверка, отчёт

Путь · Назначение

[backend/app.py](#) · HTTP API FastAPI

[backend/db.py](#) · схема и доступ к БД

[backend/detection_service.py](#) · валидация и финализация детекций

[backend/rules.py](#) · этапы, зоны, правила R-01...R-03, R-08...R-10

[backend/forecast.py](#) · прогноз, активность по серии, динамика, вердикты

[backend/plan_import.py](#) · предпросмотр CSV/XLSX плана

[backend/assistant_service.py](#) · ИИ-помощник (OpenCode Zen)

[scripts/worker.py](#) · фоновая очередь инференса
[scripts/detect_equipment_single.py](#) · запуск детектора
[frontend/](#) · Next.js 15.5 интерфейс
[migrations/001_production.sql](#) · схема PostgreSQL
[tests/](#) · unit-тесты Python
[docker-compose.yml](#), [Dockerfile*](#) · контейнерный запуск

API принимает снимок, ставит [detection_jobs](#), worker запускает локальный детектор, проверяет формат результата и атомарно пишет детекции с пересчётом правил. Статус [failed](#) не трактуется как отсутствие техники. Зависшие задания возвращаются в очередь. Инспектор подтверждает или отклоняет сигнал; вердикт по рамке — [correct](#) / [wrong](#).

Ключевые эндпоинты: [GET /api/health](#), [GET|POST /api/objects](#), [GET /api/objects/{id}](#), [POST /api/objects/{id}/upload](#), [POST /api/snapshots/{id}/detect](#), [GET /api/snapshots/{id}](#), [POST /api/warnings/{id}](#), [POST /api/detections/{id}/verdict](#), [POST /api/objects/{id}/plan/preview](#), [PUT /api/objects/{id}/stages](#), зоны и динамика объекта, [POST /api/assistant/query](#).

Переменные окружения (сервер): [BUILDWATCH_SQLITE_PATH](#), [BUILDWATCH_DATABASE_URL](#), [BUILDWATCH_DETECTOR_MODEL](#) (по умолчанию [equipment](#)), [BUILDWATCH_AUTH_REQUIRED](#), [BUILDWATCH_AUTH_SECRET](#), [BUILDWATCH_ZEN_KEY](#) / [BUILDWATCH_ZEN_MODEL](#) / [BUILDWATCH_ZEN_REASONING_EFFORT](#) для помощника. Ключ Zen не помещать в Git и во frontend.

3. Модель данных

Таблица · Значимые поля · Назначение

[objects](#) · [id](#), [name](#), [type](#), [district](#), [address](#) · Строительные объекты
[snapshots](#) · [object_id](#), [captured_at](#), [status](#), [filename](#), [src](#) · Снимки и состояние обработки
[detections](#) · [snapshot_id](#), [model](#), [label](#), [score](#), [x1...y2](#), [verdict](#) · Классы, рамки и вердикт инспектора
[stages](#) · [object_id](#), [position](#), [kind](#), [name](#), [date_from](#), [date_to](#), [status](#) · Календарные этапы
[warnings](#) · [snapshot_id](#), [rule](#), [severity](#), [why](#), [source](#), [status](#) · Проверяемые отклонения
[site_zones](#) · [object_id](#), [name](#), [kind](#), [polygon_json](#) · Зоны кадра в долях 0–1: [work](#), [danger](#), [storage](#)
[detection_jobs](#) · [snapshot_id](#), [model](#), [status](#), [attempts](#), [error](#) · Очередь инференса
[catalog](#) · [code](#), [name](#), [applies](#) · Виды работ ЛТЦ
[users](#), [audit_log](#) · пользователь, роль, действие, время · Доступ и аудит

В демо используется SQLite; для PostgreSQL — [migrations/001_production.sql](#). Кадры хранятся файлами, в БД — пути. Этап снимка: [captured_at](#) внутри [date_from](#)–[date_to](#), иначе резервно этап со статусом [current](#).

4. Детектор, правила, условия и ограничения

В контуре один детектор YOLO [equipment](#) (checkpoint [weights/equipment_v12ft_2_best.pt](#)). Веса не входят в публичный Git. Без весов UI и предзаполненное демо работают, новая детекция — нет. Инференс: SAHИ (тайлы) плюс полный кадр. Целевые восемь классов: самосвал, экскаватор, каток, кран-манипулятор, бетоносмеситель, бульдозер, грузовик, автокран. Вспомогательные классы онтологии (башенный кран, погрузчик и др.) не гарантируют одинаковое качество. Источники данных: [docs/external-training-data.md](#), [docs/mocs-dataset.md](#), [docs/openimages-training-data.md](#), [docs/ml-models.md](#).

Код · Условие · Результат

R-01 · Класс не разрешён для этапа · Сигнал о несоответствии

R-02 · Низкая уверенность или конфликт классов · Ручная проверка

- R-03 · Нет обязательной для этапа группы техники · Риск снижения темпа
- R-08 · Боксы класса почти совпадают на 2–3 снимках (IoU > 0,55) · Вопрос об активности
- R-09 · Центр бокса в полигоне **danger** · Техника в опасной зоне
- R-10 · Кран в **storage** на этапе монтажа · Вопрос инспектору

Пороги: сохранение рамок **0,15**, правила **0,35**, низкая уверенность **score < 0,5**. Для котлована ожидаются экскаватор и самосвал. Вердикт **wrong** исключает рамку из R-08...R-10 и из сравнения потребности. Прогноз считает **forecast_schedule** при чтении карточки (не пишется в **warnings**). Зоны — **POST/DELETE /api/objects/{id}/zones**. Экран качества считает вердикты инспектора, а не precision/recall модели.

Ограничения: кадры и таблицы организатора не в публичном Git (**context/dgp_extract/** нужен для локального seed); один ракурс не покрывает всю площадку; внешние holdout-метрики и MOCS/Thalos не являются оценкой текущего **equipment** v12 на московских камерах; прогноз и R-08 не доказывают физическую готовность или простой; зоны в долях кадра, не в GPS; маска обзора камеры (R-11) не реализована.

5. Сборка, установка и запуск

Windows

1. Python 3.12, Node.js 20+; **pip install -r requirements.txt**; **npm --prefix frontend ci**.
2. Для новой детекции: совместимые PyTorch и Ultralytics, файл **weights/equipment_v12ft_2_best.pt**.
3. При наличии **context/dgp_extract/**: **python scripts/seed_public_demo.py** (или **backend/seed.py / backend/demo_projects.py** для полного набора организатора).
4. **powershell -ExecutionPolicy Bypass -File scripts/start-buildwatch.ps1** — UI **http://127.0.0.1:8700**, health **http://127.0.0.1:8600/api/health**.

Linux / WSL

Из корня репозитория: **cd backend && ../.venv/bin/uvicorn app:app --host 127.0.0.1 --port 8600**; **../.venv/bin/python scripts/worker.py**; **npm --prefix frontend run dev -- --port 8700**.

Docker Compose

docker compose up --build — API, worker, frontend. Seed из **context/dgp_extract/**. Каталог **weights/** монтируется отдельно. **BUILDWATCH_ZEN_KEY** только в env сервера.

Проверки: **python -m unittest discover -s tests -v**; **npm --prefix frontend run build**; health и загрузка снимка через UI.

6. Сценарий проверки прототипа

1. Главный экран: карта портфеля, легенда, маркеры.
2. Объект с проблемой → «Открыть объект».
3. Датированный снимок и рамки; при наличии весов — загрузка нового кадра.
4. Этап на дату, ожидаемая техника, активность по серии.
5. Прогноз и фраза за 30 дней (видимая часть площадки).
6. Сигнал R-03 или R-01 с объяснением; при зоне — R-09.
7. Вердикт по сигналу, блок качества, отчёт объекта.

7. Установка камер

Камера должна перекрывать рабочие зоны и маршруты техники, сохранять время съёмки и идентификатор площадки. Нужны фиксированные ракурсы, разрешение для удалённой техники, учёт освещения и осадков. Подробности: руководство по установке камер ([camera-installation-recommendations.md](#)).